

Partie 1

Comprendre le monde dans lequel nous allons construire.



Pourquoi cette Formation ?

Dans un monde dominé par le **numérique**, comprendre Internet, l'intelligence artificielle et la création d'applications n'est plus une option.

Qu'est-ce qu'Internet réellement? C'est ça MTN/MOOV/CELTIS nous vendent quand on achète forfait internet ? **Et un site web c'est quoi?** Finalement c'est quoi la différence entre site web et internet même ? **Comment naissent WhatsApp ou Facebook? Et comment pourriez-vous créer, vous aussi?** Si vous n'avez pas de réponse, vous saisissez **l'importance de cette formation.**

Notre Vision

La **technologie** et en particulier le **numérique**, fait partie intégrante de notre vie. Comprendre son fonctionnement et l'utiliser pour réaliser des transformations durables n'est plus un choix en 2026 : **c'est devenu une obligation.**



C'est quoi un Site Web ?

On a tous déjà été sur un **site web**, enfin tant que vous avez déjà utilisé Internet. Mais concrètement, **c'est quoi un site web ?**

Un site web, de façon claire, c'est le regroupement de plusieurs **pages web**.

Ah, ce n'est peut-être toujours pas clair... Une page web alors, c'est quoi ? Une page web est un document numérique que ton navigateur (Chrome, Opera, Edge...) peut afficher. Un peu comme Word ouvre un fichier .docx pour ton exposé, ou comme une application PDF sur ton téléphone qui lit tes fiches de cours.

Donc une page web, c'est ce document que ton navigateur peut lire et t'afficher. Un site web, maintenant, c'est le regroupement de plusieurs pages web, connectées entre elles **par des liens**.

Présentons ça avec un **exemple** !

Une page web, c'est **une seule page** d'un grand livre et un site web, c'est **toutes les pages** du livre réunies, avec la couverture et tout ce qui les relie bref, c'est le livre ! Et sur un site, le lien qui connecte toutes ces pages, c'est l'hyperlien (comme la colle dans notre exemple)



Sur le site web mideessi.com une page web c'est : mideessi.com/contact et le site web c'est toutes les pages qui constituent **mideessi.com**. Mais comme un livre un site web à un but précis il peut servir à: présenter une entreprise, une personne, un produit, vendre, communiquer, distraire, etc.

C'est où un site web ?

Maintenant qu'on sait ce qu'est un site web, il faut chercher à comprendre où il se retrouve concrètement. Quand on crée un site web, où est-ce qu'on le met ? Vous avez peut-être déjà la réponse...

Plus haut nous avons affirmé:

On a tous déjà été sur un site web, enfin tant que vous avez déjà utilisé Internet.

Cela pourrait laisser croire qu'un site web (plusieurs pages web reliées par des liens) est **"dans Internet"**. Mais en réalité, ce n'est pas exactement ça !

Un site web est stocké dans une infrastructure qu'on appelle **un serveur**. Vous voyez venir ? La vraie question à se poser maintenant, c'est:

qu'est-ce qu'un serveur ? On va expliquer ce que c'est. Mais retenez qu'un site web se retrouve sur un **Serveur** !

C'est quoi un Serveur?

Un ordinateur (smartphone, tablette, PC), c'est probablement ce que vous utilisez actuellement. Sur votre appareil, vous avez des images, des vidéos, des fichiers, vos contacts, etc. Même si vous éteignez ou redémarrez votre téléphone, ces fichiers restent présents et vous pouvez les partager avec vos proches si ils vous le demande. Un **serveur**, c'est exactement la même chose : c'est un ordinateur, mais qui reste constamment allumé. Il stocke des données et les partage avec vous lorsque vous lui demandez.

Par **exemple**, lorsque vous ouvrez votre navigateur et tapez: mideessi.com

Votre **smartphone** envoie alors une demande à un autre **ordinateur** (le serveur) qui contient le site mideessi.com et souhaite le consulter. En réponse, le serveur renvoie les pages du site sur votre **smartphone**, et vous pouvez les lire grâce au navigateur.

Félicitations ! Si vous avez compris, ce sont des notions très importantes. Vous avez du mal à suivre ? Ne vous inquiétez pas, progressivement tout deviendra plus clair.



Tout va bien jusqu'ici ?

Bon, on va se calmer un peu sur les définitions et vérifier si vous avez réussi à faire une synthèse à votre niveau, et à comprendre tout ce que nous avons expliqué.

Nous allons maintenant jouer à un petit jeu: **l'Analogie**. Voici les règles! Vous vous souvenez des exemples que nous avons donnés pour représenter une page web, un site web et un serveur? Si vous avez oublié, remontez un peu plus haut pour vous rafraîchir la mémoire.

À présent, dans votre propre français, proposez d'autres exemples qui pourraient illustrer ce qu'est une page web, un site web et un serveur.

Attention: pas le droit d'utiliser l'IA, ce serait trop facile!

Écrivez vos réponses directement cliquant sur:
Selon moi.

Elles sont très importantes, car vous reviendrez sur cette page d'ici peu pour les compléter ou les mettre à jour avec plus de détails. Bon, continuons avec notre compréhension des notions de base. On se retrouve sur la page suivante.



**PASSONS A LA PAGE
SUIVANTE**

C'est quoi **Internet ?**

Ah, tu ne t'es jamais posé la question du titre de cette page ? Depuis tout à l'heure nous parlons de site web : est-ce que tu peux te rendre sur **YouTube**, par exemple, sans **Internet** ? Clairement non.

Si on te demande de définir **Internet**, pourras-tu répondre ? Si oui, alors c'est super ! Sinon, tu diras certainement : "je sais ce que c'est, mais je ne peux pas l'expliquer..." Pourtant, quand on connaît quelque chose, on doit être en mesure de l'expliquer simplement. Mais nous sommes là pour ça !

Internet, ce n'est pas ce que tu achètes chez la dame de ton quartier en disant : "Je veux acheter un forfait Internet 500 F". Tu te rappelles, plus haut, nous avons expliqué que pour afficher un site web, ton ordinateur envoie une demande à un serveur (un autre ordinateur) pour lui demander par exemple, le site mideessi.com.

Mais comment est-ce possible ?

Cela veut dire que ton ordinateur réussit à créer un lien avec un autre ordinateur que tu ne vois pas ? Et oui, c'est exactement ça ! Quand tu as une connexion Internet, ton ordinateur rejoint une immense famille de **milliers d'appareils connectés entre eux**, qui forment ce qu'on appelle **un réseau**. Donc internet est un réseau et **le plus grand réseau du monde**.

On peut alors tout simplement dire que **internet** est le regroupement de millions d'**appareils connectés entre eux**, capables d'**échanger des informations et des fichiers**.

Mais comment c'est possible ? Internet utilise des satellites, des infrastructures et de gros câbles qui relient, par exemple, l'Europe à l'Afrique. Les opérateurs mobiles (MTN, Moov, Orange) ont l'autorisation d'exploiter l'ensemble de ces infrastructures, ce qui leur permet de fournir un accès à Internet.

Quand tu achètes une connexion chez l'un d'eux, ils te donnent la possibilité de rejoindre Internet selon certaines règles: par exemple une limite de données que tu peux envoyer ou recevoir, ou une durée de validité.

Si tu prends une connexion illimitée, tu peux accéder à Internet sans restriction de volume, mais toujours selon les conditions fixées par l'opérateur.



**ON FORME UNE FAMILLE
APPELEE INTERNET !**

Comment créer un Site Web ?

Le titre de cette page est assez intéressant, parce que maintenant qu'on sait ce que c'est qu'un site web, où il reste et ces constituants, ne serait il pas temps que nous meme concevons le notre ?

Bon rappelons nous plus haut nous avons dit:

Un site web, maintenant, c'est le regroupement de plusieurs pages web, connectées entre elles par des liens.

Alors pour créer notre site que doit on faire en premier ? Je vous laisse deviner...

Evidemment si vous l'avez devinez, nous devons **créer des page web** ! Il est temps pour nous de pousser un peu nos defintions. On se retrouve sur la page suivante pour mieux approfondir tout ça.

C'est quoi une Page Web ?

Si un site web est constitué de pages web, il est logique de se demander : comment crée-t-on une page web ?

Sur ton smartphone, tu as différents fichiers : musique, vidéos, documents, etc. Ces fichiers n'ont pas tous le même type. Mais comment les distinguer ? C'est là qu'entrent en jeu les extensions.

Prenons un exemple : une chanson enregistrée sur ton téléphone. Elle a forcément un nom de fichier, par exemple :

Axel_Meryl_La_Souris_Danse.mp3

Dans cet exemple :

- Le **nom** du fichier est **Axel_Meryl_La_Souris_Danse**
- L'**extension** est **.mp3**, qui indique qu'il s'agit d'un fichier audio.

De la même manière, une page web est aussi un fichier, mais avec une extension particulière :

.html.

C'est cette extension qui permet au navigateur de reconnaître et d'afficher la page web.

Une extension est donc un petit suffixe ajouté à la fin du nom d'un fichier (après le point) qui indique son type et donc la manière dont il doit être lu ou ouvert par un programme.

Exemple :

- photo_de_toi.**jpg** → l'extension **.jpg** indique une image.
- cours_de_math.**pdf** → l'extension **.pdf** indique un document PDF.
- Axel_Meryl_La_Souris_Danse.**mp3** → l'extension **.mp3** indique un fichier audio.
- page.**html** → l'extension **.html** indique une page web.

L'extension est comme une étiquette qui dit à ton ordinateur ou ton smartphone : « Voici le type de fichier, ouvre-le avec le bon programme ». Une page web c'est juste un fichier avec l'extension **.html**

C'est quoi Html?

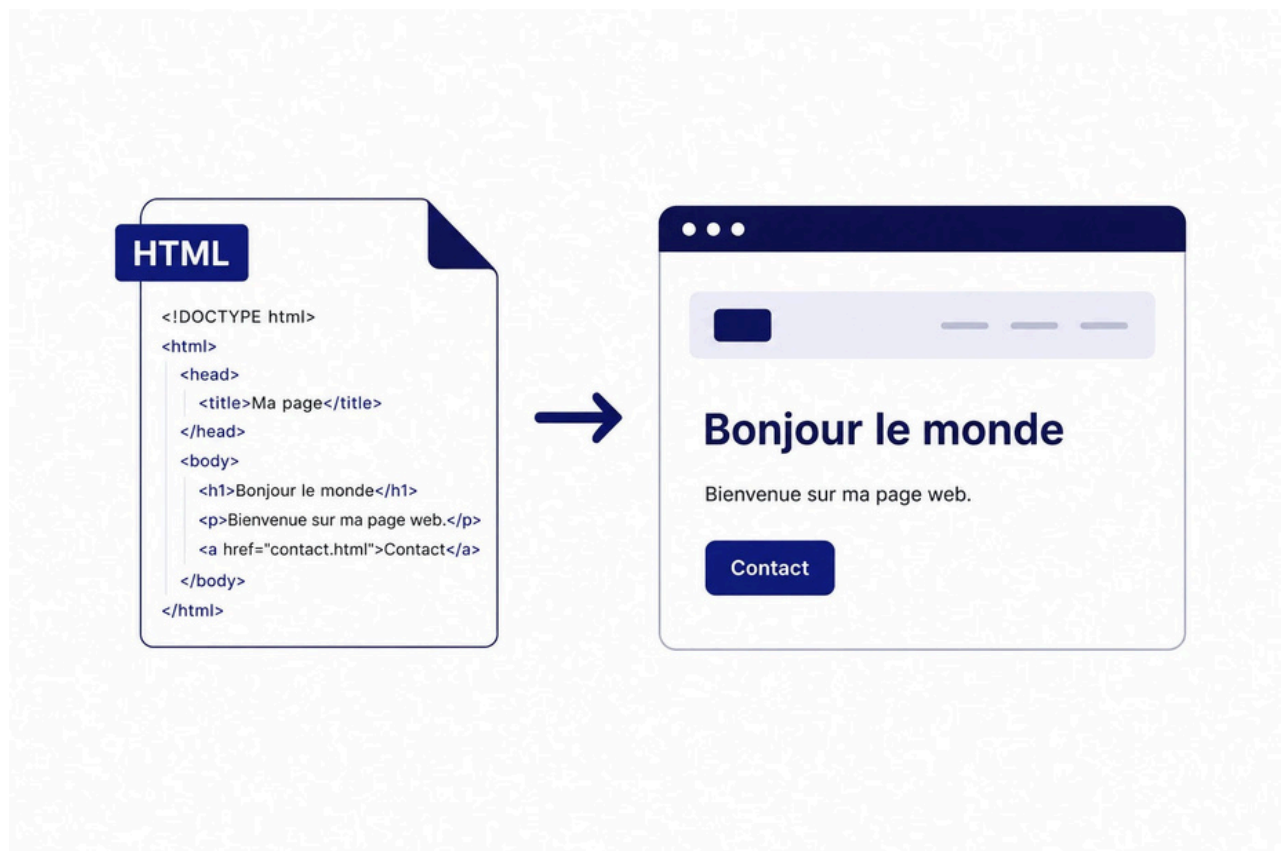
Le type de fichier qui se lit dans le navigateur est le **.html** a-t-on dit, mais il se pose une question: comment trouver un fichier de type **.html** ? Car pour avoir de la musique ou une vidéo, on sait généralement comment faire et où aller. Mais qu'en est-il du HTML ?

Alors la réponse est que, tout simplement, pour avoir du **HTML** il faut l'écrire comme un document. Tous les fichiers **.html** que tu rencontres ont été écrits dans un langage qui s'appelle le **HTML(Hypertexte Markup Language)** . C'est un langage de **balisage** qui permet de structurer le contenu d'une page web.

Structurer ?

Eh ben oui, lorsqu'on écrit un document Word ou un texte, on met bien des titres qui sont les plus grands, des sous-titres qui sont moyens, et ensuite des paragraphes, les plus petits.

C'est pareil pour une page web. Le **HTML** permet de dire au navigateur : ceci est un titre, ceci est un paragraphe, et le navigateur sait comment l'afficher. On part alors d'un fichier **HTML**, qui s'écrit dans un éditeur de code, et ce fichier est affiché dans un navigateur en tant que page web.



LA MAGIE S'OPÈRE

Où écrire du **HTML**?

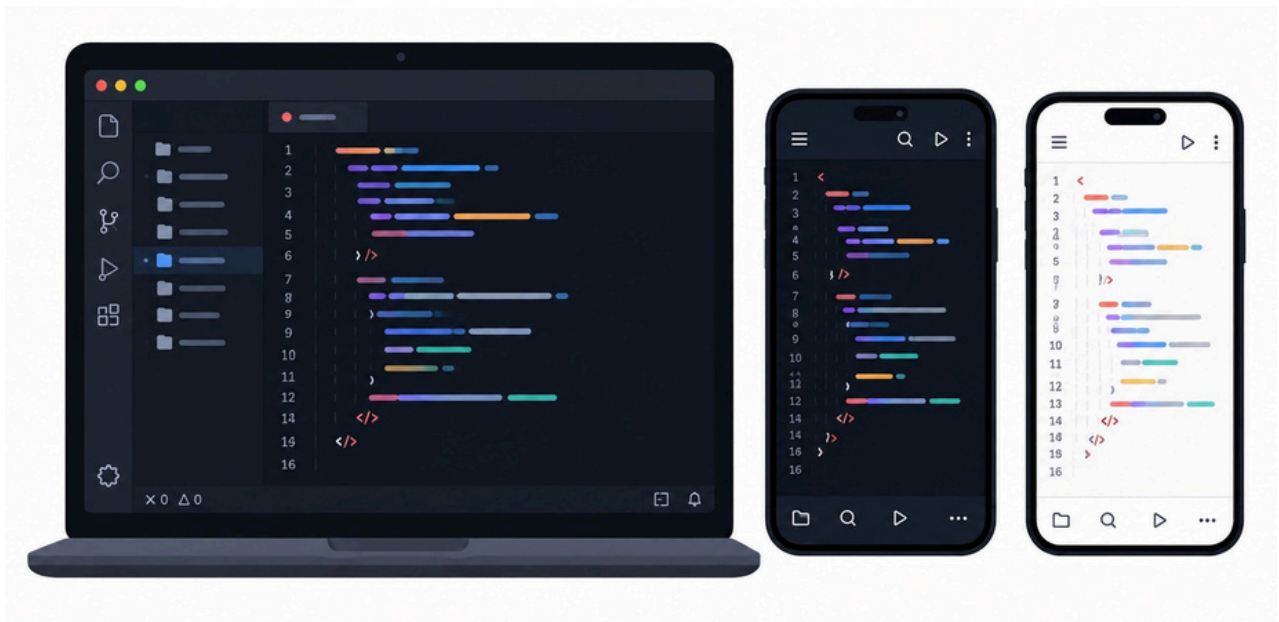
Après avoir compris ce qu'était le **HTML**, la suite logique est donc de savoir où et comment l'écrire, mais nous allons nous concentrer sur le "où" avant de nous lancer dans le "comment". Donc où écrire du **HTML** ?

Le **HTML** peut s'écrire dans un éditeur de texte comme ton bloc-notes, mais il est généralement conseillé de travailler dans un environnement adapté, notamment les **éditeurs de code**, qui sont plus pratiques car ils colorent ton code et t'aident à repérer les erreurs.

Un éditeur de code est un logiciel qui est conçu spécialement pour écrire et modifier du code informatique donc on peut y écrire du HTML tout comme d'autres langages comme CSS, JS etc. Il permet également de styliser le code en y ajoutant de la couleur, les plus récents permettent une intégration de l'IA et des outils comme Git ou Github et d'aider pour le développement.

Vous comprenez donc pourquoi il est mieux de coder dans un éditeur de code. Les plus populaires sont gratuits, comme **Visual Studio Code** ou **Sublime Text** sur PC, ou **Acode**, **SPCK Editor** sur smartphone. Et oui, on peut coder avec un smartphone.

Nous te recommandons alors d'installer un éditeur de code pour suivre correctement la prochaine étape. Il est généralement très facile de l'installer, et de nombreux tutoriels sont disponibles si tu es bloqué.



MON CODE PARTOUT !

Comment écrire du **Html**?

Pour débiter on suppose que vous aviez déjà installé Visual Studio Code, ainsi nous allons créer notre premier fichier dans Visual Studio Code. Plangeons dans le bain.

Nous allons vous demander d'ouvrir Visual Studio Code et de créer votre premier fichier en HTML. Une fois le fichier totalement vide créé veuillez entrer ceci :

Je commence le HTML
Merci à MIDEESI LEARN

Après ceci nous allons enregistrer notre fichier mais lors de l'enregistrement au lieu de mettre pas juste un nom comme **monfichier** Nous allons ajouter l'extension **.html** pour préciser que c'est un fichier de type HTML. Ce qui donnera **monfichier.html**. Maintenant, ouvre ce fichier .html avec ton navigateur. Que remarques-tu ?

Ton texte s'affiche bien, mais tout à la suite, sur une seule ligne ou en bloc, sans aucune mise en forme. Le navigateur ne sait pas que ta première ligne devait être un titre, par exemple.

Malgré que ce fichier soit un fichier avec l'extension **.html**, la page s'affiche comme un simple texte. C'est normal, nous avons fait ce petit exercice pour vous montrer réellement l'importance de **HTML**. Pour donc aider le navigateur à distinguer les différentes parties d'un fichier (titre, sous-titres, images) on utilise **des balises**. Nous approfondissons dans la partie suivante

C'est quoi une balise ?

On vous répète depuis le début qu'HTML sert à structurer, HTML sert à organiser, HTML sert à hiérarchiser, HTML sert à donner du sens au contenu d'une page web. Mais comment ça se passe réellement ?

Une balise est un élément du langage **HTML** qui sert à structurer et à donner du sens au contenu d'une page web.

Concrètement, une balise est un mot-clé entouré de chevrons (◀ ▶) que le navigateur (Chrome, Firefox...) sait reconnaître et interpréter. Quand le navigateur lit une balise, il comprend immédiatement quel rôle joue le contenu qui l'accompagne : est-ce un titre ? un paragraphe ? une image ? un lien ?

Comment écrire une balise ?

Connaître ce qu'est une balise sans pouvoir l'écrire n'aide pas vraiment dans notre apprentissage donc nous allons apporter la réponse à cette question **Comment écrire une balise ?**

Une balise HTML fonctionne par paire: une balise d'ouverture, un contenu, puis une balise de fermeture. La balise de fermeture reprend toujours le même nom que la balise d'ouverture, avec un / ajouté juste après le <

```
<nom-de-la-balise>Contenu</nom-de-la-balise>  
<h1>Je commence le HTML</h1>
```

Il ne faut pas oublier de fermer les balises sinon cela risque de casser ton code et de causer des bugs.

Il est obligatoires de fermer les balises mais pas toutes les balises car certaines n'en ont pas besoin. Il s'agit des **balises orphelines (ou "auto-fermantes")**. Les balises orphelines (ou auto-fermantes) sont des balises HTML qui n'ont pas besoin d'une balise de fermeture, car elles ne contiennent pas de contenu interne. Il existe quelques unes que nous allons utiliser énormément dans notre suite.

**
**

<hr>

Prenons l'exemple du chapitre passé on avait écrit dans un fichier HTML mais dans le navigateur le résultat c'était affiché comme un simple texte parce qu'on avait utilisé de balise.

Nous allons réessayer avec des balises pour voir la différence. Retournons dans notre fichier monfichier.html et remplaçons le contenu par celui ci :

`<h1>`Je commence le HTML`</h1>`
`<p>`Merci à MIDEESSI LEARN`</p>`

Si vous ouvrez le navigateur, vous verrez que les balises ne s'affichent pas, seul le contenu qu'elles contiennent apparaît à l'écran. Et aussi elles ne sont pas pareilles. Par exemple, le titre **`<h1>`** s'affiche plus grand et plus gras que le paragraphe **`<p>`**. Le langage est composé uniquement de balises donc sans balises pas de HTML, il en existe plusieurs balises indispensables pour écrire du html que nous verrons dans la suite. Une balise s'ouvre avec **`<nom>`** et se ferme avec **`</nom>`**



C'EST SIMPLE !

Structure d'une page HTML

Jusqu'ici nous avons écrit du code dans un fichier **.html** vide et cela s'affichait correctement mais en réalité ce n'est pas encore du bon HTML. Tous les fichiers html ont une structure de base général. Quel est cette structure ? C'est ce qu'on découvre maintenant.

Lorsqu'une page web est créée, elle est consultée par plusieurs personnes, sur des appareils et des navigateurs différents. Pour qu'elle s'affiche correctement partout, chaque page doit respecter une organisation précise c'est là qu'intervient la structure de base d'une page HTML. La structure sert à indiquer au navigateur la version du langage utilisé, afin qu'il sache comment interpréter le code qui suit. Elle garantit aussi que la page s'affiche correctement partout, puisqu'une structure respectée permet un rendu identique sur Chrome,

Firefox, ou le navigateur d'un téléphone Android, aide Google à comprendre de quoi parle la page, ce qui lui permet de mieux la classer dans les résultats de recherche. Nous allons à présent voir la structure de base d'une page html.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Ma première page</title>
  </head>
  <body>
    <h1>Bienvenue !</h1>
    <p>Ceci est mon premier paragraphe.</p>
  </body>
</html>
```

Voici la structure de base d'une page HTML. Selon l'éditeur de code, lorsque tu crées un fichier HTML, elle peut être générée automatiquement. Mais si ce n'est pas le cas, tu devras l'écrire obligatoirement avant de commencer. Mais que veux dire ces éléments ?

Nous allons décortiquer chacun des éléments de cette structure. Déjà tu peux constater que cette structure est composée uniquement de balise comme on l'avait dit. On peut déjà voir qu'il y'a la balise **<!DOCTYPE html>**, cette balise est la première de tout document HTML moderne, elle permet de dire au navigateur sur la version d'HTML dont il s'agit, ici c'est le HTML5. Il est nécessaire car c'est en fonction de la version d'HTML que le navigateur sait comment il doit lire le fichier.

La balise <html>

C'est la racine du document. Tout ce que tu écris dans ta page doit être placé entre <html> et </html>. Elle sert à dire au navigateur: «Voici le début et la fin de mon document HTML.»

La balise <head>

Cette partie est invisible pour l'utilisateur. Elle contient des informations importantes comme le titre de la page, les métadonnées, ou encore les liens vers des fichiers CSS et JavaScript. On approfondira cette partie plus loin.

La balise **<title>**

Elle se place dans le `<head>`. Son rôle est de définir le nom de la page qui apparaît dans l'onglet du navigateur. Par exemple, si tu écris **<title>**Ma première page**</title>**, tu verras ce texte dans l'onglet en haut de ton navigateur.

La balise **<body>**

C'est la partie visible de la page. Tout ce que tu veux montrer à l'utilisateur (titres, paragraphes, images, vidéos, liens, etc.) doit être placé entre **<body>** et **</body>**. Donc tout ce qu'on va écrire désormais devra se placer dans le body.

Balises pour le **texte**

Nous avons vu les **balises de structures** qui sont obligatoires pour écrire un fichier HTML mais ce sont les seules qu'on utilise en HTML? Bien sûr que **NON!**

À part ces balises, il existe les **balises de contenu** qui permettent d'afficher et de structurer les éléments qui vont s'afficher dans le navigateur notamment **du texte, des images et des liens**. C'est elle qui permettent de structurer ce que l'utilisateur voit dans le navigateur. Il n'est donc inutile de rappeler que ces balises seront mis dans le body. Quelles sont ces balises de contenus ?

Y'en a plusieurs qui ont chacune leur rôle bien précis. HTML propose 6 niveaux de titres, numérotés de 1 à 6 : **<h1>**, **<h2>**, **<h3>**, **<h4>**, **<h5>** et **<h6>**. Et pour le texte normal, il y a la balise **<p>**.

Tu as sûrement déjà remarqué que dans un document Word (ou dans n'importe quel livre), il y a différents niveaux de titres : un titre principal, des sous-titres, puis le texte normal (les paragraphes). On ne choisit pas qu'un texte est un titre parce qu'il est gros, on le déclare comme titre parce qu'il représente une section importante.

HTML fonctionne exactement de la même façon. Quand tu écris **«h1»**, tu ne dis pas au navigateur "affiche ce texte en gros", tu lui dis "ceci est le titre le plus important de la page". C'est le navigateur qui décide, tout seul, comment le montrer à l'écran : il sait que **«h1»** est le niveau le plus important, alors il l'affiche avec la plus grande taille. Quand tu écris **«h2»**, tu dis "ceci est important, mais un peu moins que le h1" et le navigateur l'affiche un peu plus petit. Et ainsi de suite, jusqu'à **«h6»**, le niveau le moins important, que le navigateur affiche presque comme du texte normal.

Et le texte normal, justement, celui qui n'est ni un titre ni un sous-titre, juste une phrase ou un texte classique, ça s'écrit avec **«p»**.

Voici un petit recapitulatif des titres et sous-titres :

<h1>Titre principal</h1>
<h2>Titre de section</h2>
<h3>Sous-section</h3>
<h4>Sous-sous-section</h4>
<h5>Détail</h5>
<h6>Petit détail</h6>
<p>Un texte normal.</p>

Maintenant après avoir beaucoup parlé sur les titres et sous titres nous allons passer à un peu de pratiques. Nous allons essayer de reproduire une page web en partant du resultat qui s'afficge dans le navigateur. En repartant dans le fichier que nous avons crée avant ou en créant un nouveau fichier .html tu vas écrire le code qui nous donne ce résultat. Rappelle toi comment s'écrit une balise.

MIDEESSI Learn

Nos formations

Développement web pour débutants

Une formation pensée pour les débutants complets, avec des exemples ancrés dans le quotidien béninois pour apprendre à créer ses premières pages web.

Une balise **h1** ne s'écrit qu'une seule fois par page, car elle représente le titre principal, et une page n'a qu'un seul titre principal.

Les balises h1 à h6 ne servent pas à mettre du texte en gras, même si le navigateur les affiche ainsi par défaut. Ce n'est pas leur rôle. Le HTML est un langage de structure : il sert à organiser et hiérarchiser le contenu d'une page, pas à le styliser.

On ne met donc jamais un élément en h1 parce qu'on veut qu'il soit gros et gras, mais parce qu'il représente le titre le plus important de la page. On est dans un h1 parce que c'est important, ce n'est pas important parce que c'est dans un h1.

Les listes.

Après avoir appris à structurer nos pages avec des titres et des paragraphes, il est temps de découvrir un autre outil très utile: les listes. Elles permettent de présenter des informations de manière ordonnée ou hiérarchisée, ce qui rend le contenu plus lisible et plus facile à comprendre. Comme toujours en HTML, nous allons utiliser des balises spécifiques: pour les listes, il en existe trois principales: ****, **** et ****. À ce stade, tu te demandes sûrement comment ces balises vont nous permettre d'écrire des listes... allons le découvrir ensemble!

En HTML, il existe deux types de listes: les listes ordonnées et les listes non ordonnées.

- **Les listes ordonnées**

Elles permettent de présenter des éléments dans un ordre précis (par exemple, un classement ou des étapes à suivre). Pour les créer, on utilise la balise **** (qui signifie **ordered list**), et à l'intérieur, chaque élément est placé dans une balise **** (**list item**).

Étape 1 : Ouvrir le fichier

Étape 2 : Écrire le code

Étape 3 : Enregistrer

Vous pouvez constater que, dans une liste ordonnée, les informations s'affichent automatiquement avec des numéros. Cela permet de présenter les éléments dans un ordre précis, comme des étapes ou un classement.

1.Étape 1 : Ouvrir le fichier

2.Étape 2 : Écrire le code

3.Étape 3 : Enregistrer

Résultat que vous devriez obtenir dans le navigateur si vous tapez le code que nous avons écrit plus haut.

- **Les listes non ordonnées**

Elles servent à présenter des éléments sans ordre particulier, comme une liste de courses ou d'idées. Pour cela, on utilise la balise `<ul>` (unordered list), et là encore, chaque élément est placé dans une balise `<li>`.

```
<ul>  
  <li>Pomme</li>  
  <li>Banane</li>  
  <li>Orange</li>  
</ul>
```

Par contre, vous verrez que les éléments ne sont pas numérotés. Ils apparaissent simplement avec des puces, car il s'agit d'une liste non ordonnée.

- **Pomme**
- **Banane**
- **Orange**

Les liens.

Au fur et à mesure que nous avançons, notre page web se remplit et peut devenir très longue, avec parfois des centaines de lignes de code. Nous avons déjà comparé un site web à un livre : chaque page web est comme une page de ce livre. Dans un livre, les pages sont reliées entre elles par une reliure ; mais sur le web, comment plusieurs pages peuvent-elles être associées pour permettre la navigation ? La réponse est simple : grâce aux liens. Les liens permettent de relier nos différentes pages, de passer de l'une à l'autre et de naviguer facilement dans l'ensemble du site. Ils sont donc essentiels pour donner vie à la structure d'un site et offrir une véritable expérience de lecture et de navigation.

Un lien permet de rendre un texte cliquable et de diriger l'utilisateur vers une autre page ou une autre ressource. C'est grâce aux liens que l'on peut naviguer facilement d'une page à l'autre et donner vie à la structure d'un site web. Et comment écrit-on un lien ?

Les liens en HTML se créent avec la balise <a> (anchor).

- L'attribut principal est href (Hypertext REFerence), qui indique la destination du lien.

On commence à styliser

LOT 02



Étape 1

- Crée un projet avec un fichier HTML et un fichier CSS, reliés par la méthode recommandée.
- Construis la structure suivante : un en-tête avec un titre principal et une liste de navigation de 3 liens ; une zone principale contenant une section qui regroupera bientôt plusieurs articles ; un pied de page avec un paragraphe de mention.
- Choisis toi-même les classes utiles pour la suite, en anticipant ce que tu vas devoir cibler séparément.
- Donne à l'en-tête et au pied de page des couleurs de fond différentes l'une de l'autre, en ciblant précisément chaque zone sans affecter le reste.
- Modifie une des couleurs uniquement dans le CSS et vérifie que ça s'applique sans toucher au HTML.

Étape 2

- Dans la section principale, ajoute 3 articles, chacun avec un titre et un paragraphe.
- Tous les paragraphes de la page doivent être gris, sauf ceux à l'intérieur d'un article, qui doivent être bleu foncé sans toucher au HTML.
- Un seul des trois articles doit se distinguer visuellement des autres, sans affecter les deux autres.
- Les liens de navigation ne doivent pas être soulignés et doivent tous partager la même couleur, en une seule règle groupée.

Étape 3

- Ajoute, dans un article différent de celui déjà mis en avant à l'étape 2, un paragraphe signalant une info importante. Il doit s'afficher en rouge malgré la règle sur les paragraphes d'article sans **!important**.

- Donne à un des liens de navigation un identifiant unique. Ce lien doit garder une couleur fixe même si une classe tente ensuite de la changer.
- Ajoute une règle sur le body qui définit une police et une couleur pour toute la page. Vérifie où le contenu en hérite, et identifie au moins un endroit de ta page où cet héritage ne s'applique pas, à cause d'une règle plus spécifique.
- Ajoute une bordure sur la section qui contient les articles, et vérifie qu'elle ne se transmet pas aux articles.

Etape 4

- La section contenant les articles doit occuper 90% de la largeur de la page, centrée.
- Chaque article doit faire environ un tiers de la largeur de cette section, moins un peu d'espace entre eux en pourcentage.
- Le titre principal doit avoir une taille définie par rapport à la taille de base du document, et les titres d'articles une taille définie par rapport à leur parent direct choisis l'unité adaptée à chaque cas.

- Change la taille de police du html et vérifie que le titre principal réagit, mais que les titres d'articles dépendent de leur propre parent.
- Reviens sur l'en-tête et le pied de page de l'étape 1 : ajuste leur espacement intérieur et leur couleur de texte pour que l'ensemble ait un rendu visuellement cohérent c'est le résultat concret final attendu.

Etape 5

- Reprends les couleurs "de base" posées à l'étape 1 sur l'en-tête et le pied de page, et remplace-les par des valeurs hexadécimales précises, plus par des noms de couleur.
- Le fond de l'en-tête doit utiliser `rgba()` avec une transparence partielle.
- L'article mis en avant (étape 2) doit avoir un fond différent des deux autres articles, avec une couleur de texte qui reste lisible dessus ne remets pas le même bleu foncé que les autres articles si le contraste ne fonctionne pas.
- Le paragraphe d'alerte de l'étape 3 doit désormais utiliser `rgba()` pour un rouge semi-transparent, plutôt qu'un rouge plein.

- Change la taille de police du html et vérifie que le titre principal réagit, mais que les titres d'articles dépendent de leur propre parent.
- Reviens sur l'en-tête et le pied de page de l'étape 1 : ajuste leur espacement intérieur et leur couleur de texte pour que l'ensemble ait un rendu visuellement cohérent c'est le résultat concret final attendu.

Etape 5

- Reprends les couleurs "de base" posées à l'étape 1 sur l'en-tête et le pied de page, et remplace-les par des valeurs hexadécimales précises, plus par des noms de couleur.
- Le fond de l'en-tête doit utiliser `rgba()` avec une transparence partielle.
- L'article mis en avant (étape 2) doit avoir un fond différent des deux autres articles, avec une couleur de texte qui reste lisible dessus, ne remets pas le même bleu foncé que les autres articles si le contraste ne fonctionne pas.
- Le paragraphe d'alerte de l'étape 3 doit désormais utiliser `rgba()` pour un rouge semi-transparent, plutôt qu'un rouge plein.

Étape 6

- Choisis une police pour le titre principal, différente de celle du reste du texte (font-family, avec une police de secours générique dans les deux cas).
- Le titre principal doit être nettement plus épais que le texte courant (font-weight), et les titres d'articles doivent avoir un poids intermédiaire entre les deux.
- Les paragraphes des articles doivent avoir un interlignage confortable (line-height) ; le pied de page, plus court, peut rester plus compact.
- Le lien de navigation qui a une couleur fixe depuis l'étape 3 doit aussi se démarquer par un style italique.
- Le titre principal doit être centré, les titres d'articles alignés à gauche, le pied de page justifié.

Étape 7

- Ajoute une image de fond sur l'en-tête (un chemin même fictif suffit), positionnée pour ne pas se répéter et couvrir toute la zone sans être déformée.
- L'article mis en avant doit avoir son propre fond en image plutôt qu'en couleur unie, tout en gardant le texte lisible par-dessus, à toi de décider comment combiner couleur et image si besoin.
- Vérifie qu'aucune image ne se répète en mosaïque ailleurs sur la page par accident, à cause du comportement par défaut du navigateur..

Étape 8

- Ajoute un padding intérieur généreux à chaque article, pour que le texte ne colle pas aux bords — mais fais attention à ne pas casser les largeurs en pourcentage fixées à l'étape 4 (pense à box-sizing).

Ajoute une bordure fine sur chaque article (différente de celle déjà posée sur la section à l'étape 3).

- Gère l'espacement entre les trois articles avec du margin, et vérifie si un phénomène de fusion des marges apparaît entre l'en-tête et la section qui suit, corrige-le si besoin.
- Applique box-sizing: border-box sur l'ensemble de la page via le sélecteur universel, pour que padding et bordure ne fassent plus dépasser tes largeurs en pourcentage.
- Revois le padding et la marge de l'en-tête et du pied de page pour que l'ensemble de la page respire, plutôt que du contenu collé aux bords de la fenêtre.

Exercice 2

Partiel

Recherche : En CSS, plusieurs propriétés définissent l'apparence de base d'un texte. Cherche à répondre à ces questions :

- A quoi sert **font-family**, et pourquoi indique-t-on souvent plusieurs polices à la suite (police principale, police de secours, famille générique) ?
- À quoi sert **font-size**, et quelle différence y a-t-il entre une taille en **px** et une taille en **rem** ?
- À quoi sert **font-weight**, et quelles valeurs peut-il prendre ?

Travaux pratiques

1. Créer un fichier HTML avec un titre (h1) ayant une classe (par exemple class="titre"), lié à un fichier CSS.
2. Cibler cette classe pour définir une police avec **font-family**, en indiquant une police principale, une police de secours, et une famille générique.

3. Toujours via la même classe, définir une taille avec **font-size** en px, puis la remplacer par une taille équivalente en rem.
4. Ajouter **font-weight** et tester une valeur en mot (**bold**) puis une valeur numérique (par exemple **600**).

Partie 2

Recherche : En CSS, le texte n'est pas seulement du contenu brut: il peut être aligné et décoré pour améliorer la lisibilité et l'esthétique d'une page web. Cherche à répondre à ces questions :

- À quoi sert **text-align**, et quelles valeurs peut-il prendre ?
- À quoi sert **text-decoration**, et quelles valeurs peut-il prendre ?

Travaux Pratiques:

1. Créer un fichier HTML avec trois paragraphes distincts (par exemple une adresse, une consigne, et un avertissement), chacun avec sa propre classe. Lier ce fichier à un fichier CSS.

2. Sur le premier paragraphe, appliquer **text-align: left**. Sur le deuxième, **text-align: center**. Sur le troisième, **text-align: right**. Observer la position de chaque texte dans la page.
3. Ajouter un lien (a) dans le fichier HTML, avec une classe (par exemple class="lien"), et lui appliquer **text-decoration: underline**.
4. Sur ce même lien, remplacer par **text-decoration: line-through**, puis par **text-decoration: none**, en observant le changement à chaque étape.

Exercice 3

Partiel

Recherche : En CSS, la propriété **background-color** permet de définir la couleur de fond d'un élément HTML. C'est l'une des premières façons de donner du style à une page web, car elle aide à mettre en valeur le contenu et à rendre l'interface plus agréable visuellement. Cherche à répondre à cette question :

- A quoi sert **background-color**, et quels formats de valeur peut-elle prendre (nom, hexadécimal, rgb())

Travaux pratiques

1. Créer un fichier HTML avec un conteneur (div) ayant une classe (par exemple class="bloc"), lié à un fichier CSS. Donner à ce conteneur une largeur et une hauteur fixes (par exemple 300px sur 200px) pour bien voir le fond.
2. Cibler cette classe pour lui appliquer une background-color avec un nom de couleur.

3. Remplacer cette valeur par un code hexadécimal, puis par un `rgb()`, en observant que le résultat peut être identique selon les valeurs choisies.

Partie 2

Recherche : En CSS, la propriété `background-image` permet d'afficher une image en arrière-plan d'un élément HTML. C'est un outil essentiel pour donner du style à une page et créer des visuels attractifs. Cherche à répondre à ces questions :

- A quoi sert **`background-image`**, et comment s'écrit la valeur avec `url()` ?
- Quelle différence y a-t-il entre un chemin relatif (par exemple `images/photo.jpg`) et un chemin absolu
- Pourquoi faut-il vérifier que le chemin relatif correspond bien à l'emplacement réel du fichier ?

Travaux pratiques

1. Créer un dossier contenant un fichier HTML, un fichier CSS, et une image (par exemple dans un sous-dossier images).
2. Créer un conteneur (div) avec une classe (par exemple `class="fond-image"`), une largeur et une hauteur fixes, lié au fichier CSS.
3. Cibler cette classe pour lui appliquer **background-image** avec `url()`, en utilisant le chemin relatif vers l'image.
4. Vérifier que l'image s'affiche bien. Si elle ne s'affiche pas, vérifier que le chemin écrit correspond exactement à l'emplacement du fichier image.

Exercice 4

Partiel

Recherche : En CSS, chaque élément HTML est représenté comme une boîte. Comprendre ce modèle de boîte est essentiel pour bien maîtriser la mise en page et l'espacement. Cherche à répondre à ces questions :

- Qu'est-ce que le contenu d'une boîte en CSS ?
- À quoi sert le **padding**, et entre quoi se situe-t-il exactement ?
- À quoi sert la **border**, et quelles informations faut-il lui donner (épaisseur, style, couleur) ?

Travaux pratiques

1. Créer un fichier HTML avec un conteneur (div) contenant un court texte, avec une classe (par exemple class="boite"), lié à un fichier CSS.
2. Cibler cette classe pour donner au conteneur une couleur de fond (par exemple avec background-color), afin de bien voir ses limites.

3. Ajouter un padding (par exemple 20px) et observer l'espace qui apparaît entre le texte et le bord coloré de la boîte.
4. Ajouter une border (par exemple 2px solid black) et observer qu'elle s'ajoute autour du padding.

Partie 2

Recherche : Cherche à répondre à ces questions :

- A quoi sert le margin, et en quoi est-il différent du padding ?
- Le margin fait-il partie de la boîte visible (avec sa couleur de fond) ou est-il à l'extérieur ?

Travaux pratiques

1. Créer un fichier HTML avec deux conteneurs (div) l'un après l'autre, chacun avec une classe (par exemple class="boite1" et class="boite2"), tous deux avec une couleur de fond différente. Lier le fichier à un CSS.

2. Sans margin, observer que les deux boîtes se touchent.
3. Ajouter un margin (par exemple 30px) sur le premier conteneur et observer l'espace qui apparaît maintenant entre les deux boîtes, à l'extérieur de la couleur de fond.

Les bonnes Manière

Avant d'aller plus loin dans cette formation, il est important de vous présenter quelques réflexes et attitudes à adopter lorsque vous codez. Nous allons donc commencer par la hiérarchie visuelle de votre code. Regardez le contenu de deux pages web juste en dessous.

Page 1

```
<!DOCTYPE html>
<html>
<head>
<title>Ma Page</title>
</head>
<body>
  <div class="container">
<h1>Bienvenue</h1>
    <p>Voici un exemple de contenu.
    </p>
    <ul>
      <li>Premier élément</li>
      <li>Deuxième élément</li>
      <li>Troisième élément</li>
    </ul>
    <div class="card">
      <h2>Carte 1</h2>
      <p>Description de la carte.</p>
    </div>
    <div class="card">
      <h2>Carte 2</h2>
      <p>Autre description.</p>
    </div>
  </div>
</body>
</html>
```

Page 2

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Ma Page</title>
</head>
<body>
  <div class="container">
    <h1>Bienvenue</h1>
    <p>Voici un exemple de contenu.</p>

    <ul>
      <li>Premier élément</li>
      <li>Deuxième élément</li>
      <li>Troisième élément</li>
    </ul>

    <div class="card">
      <h2>Carte 1</h2>
      <p>Description de la carte.</p>
    </div>

    <div class="card">
      <h2>Carte 2</h2>
      <p>Autre description.</p>
    </div>
  </div>
</body>
</html>
```

Ces deux blocs de code produisent exactement le **même résultat** à l'écran. Ouvrez-les dans un navigateur, et vous verrez le même titre, le même paragraphe, la même liste, les deux mêmes cartes. Aucune différence visuelle pour l'utilisateur final.

Et pourtant, si vous deviez travailler sur l'un de ces deux fichiers dans six mois, ou pire, si un collègue devait le reprendre sans vous à côté, la différence serait énorme (parce que oui on code bien souvent sur un même projet à plusieurs).

Avant d'aller plus loin, il faut clarifier un mot: **l'indentation**.

L'indentation, c'est tout simplement le fait de **décaler une ligne de texte** vers la droite, en ajoutant des **espaces** (ou une tabulation) au début de cette ligne. Vous l'avez déjà vu sans le savoir: quand vous rédigez un document Word et que vous commencez un nouveau paragraphe en laissant un petit retrait au début de la première ligne, c'est de l'indentation.

Dans le code, on utilise ce même principe, mais de façon beaucoup plus systématique : chaque fois qu'un élément est "contenu" dans un autre, on le décale d'un cran vers la droite pour le montrer visuellement.

MaPetiteFamille

Papa

Maman

Ma soeur

Moi

La deuxième ligne "Papa" est **indentée** : elle commence plus loin que la première, parce qu'on a ajouté des espaces avant le mot. En **HTML**, on utilise cette technique pour montrer qu'une balise se trouve à l'intérieur d'une autre. Plus une balise est "profonde" dans la structure de la page, plus elle est décalée vers la droite.

Revenons maintenant à nos deux versions.

Dans la Version 1, l'indentation est utilisée n'importe comment. Certaines lignes sont décalées, d'autres non, sans aucune logique.

Votre œil doit alors faire tout le travail: il n'y a aucun repère visuel fiable pour savoir quelle balise est contenue dans quelle autre.

Le **<h1>** semble flotter au même niveau que le **<div class="container">** qui le contient, alors qu'il est censé être à l'intérieur.

Les deux **<div class="card">** n'ont pas la même indentation alors qu'ils jouent exactement le même rôle. Pour comprendre la structure réelle du document, vous êtes obligé de la reconstruire mentalement, balise par balise, en comptant les ouvertures et les fermetures.

Dans la Version 2, l'indentation suit une règle simple et constante: chaque fois qu'une balise en contient une autre, le contenu est décalé d'un cran vers la droite (ici, deux espaces par niveau).

Ce geste a l'air simple, mais pourtant c'est l'une des meilleures attitudes à adopter en codant. Vous serez toujours plus motivé à travailler dans un fichier propre et bien organisé que dans un fichier écrit en désordre et sans organisation visuelle.

En plus de l'indentation, il existe une autre pratique tout aussi importante: **les commentaires**. Nous avons déjà mentionné qu'un projet peut durer plusieurs mois et impliquer plusieurs personnes. Il arrive un moment dans le développement où l'on ne se rappelle plus pourquoi on a fait tel choix: pourquoi j'ai utilisé cette balise et pas une autre, etc. Chaque développeur a sa propre méthode pour coder, et ton collègue ne peut pas forcément comprendre tes intentions. Pour éviter ces problèmes, on ajoute de petits bouts de texte dans le code afin d'expliquer nos choix. Ces explications ne sont pas affichées dans le navigateur.

Vous vous demandez sûrement comment écrire un commentaire?

Un commentaire en HTML s'écrit toujours entre `<!--` et `-->` : tout ce qui se trouve entre ces deux marqueurs devient invisible pour l'utilisateur et n'est jamais affiché par le navigateur.

```
<!-- Ceci est un commentaire -->
```

```
<h1>Bienvenue sur MIDESSI</h1>
```

Si vous tapez ce code et ouvrez le navigateur, vous verrez que le commentaire ne s'affiche pas. Dès que vous ouvrez `<!--`, tout ce qui suit sera considéré comme un commentaire. Assurez-vous donc de bien le refermer avec `-->`.

On ne met pas des commentaires partout. Oui, cela sert à expliquer, mais il n'est pas conseillé d'ajouter un commentaire à chaque ligne. On en met uniquement lorsque c'est flou ou que le code n'est pas immédiatement compréhensible. Par exemple, on ne va pas mettre un commentaire pour expliquer une balise `<h1>` : tout le monde sait à quoi elle sert.

Il faut donc trouver le juste milieu : assez de commentaires pour rendre le code lisible et compréhensible, mais pas trop pour éviter de l'alourdir inutilement.

Du code Propre

En codant, on remarque souvent que plusieurs balises tournent autour d'un même thème, ou décrivent une seule et même chose. Par exemple : un titre pour le nom d'une entreprise, une image pour son logo, et un paragraphe pour la présenter. Individuellement, ce sont des balises séparées, `<h2>`, `<img>`, `<p>`, mais ensemble, elles racontent une seule histoire : "**qui est cette entreprise ?**"

Le problème, c'est qu'en HTML, rien ne dit au navigateur que ces balises vont ensemble. Elles sont juste... les unes à côté des autres, sans lien entre elles. On a donc besoin d'une balise capable de les regrouper, pour dire clairement : "ces éléments ont un lien" C'est exactement le rôle des conteneurs.

Et ce regroupement peut se faire de deux façons : soit par thème, comme notre exemple sur l'entreprise, soit par emplacement dans la page, comme regrouper tout ce qui constitue l'en-tête ou le pied de page etc. Ce sont les **conteneurs**

Un conteneur, c'est une balise qui regroupe plusieurs éléments ensemble pour former un seul bloc logique. Il n'affiche rien par lui-même, son seul rôle est de rassembler du contenu déjà existant. Il en existe plusieurs, et la différence entre eux, c'est ce qu'ils disent sur leur contenu

Certains conteneurs regroupent des éléments sans rien dire sur eux, comme **«div»** ou **«span»**. On les appelle des **conteneurs génériques**, car ils restent muets sur ce qu'ils contiennent. D'autres, au contraire, disent clairement ce qu'ils représentent, comme **«header»**, qui annonce "je suis l'en-tête de la page", ou **«footer»**, qui annonce "je suis le pied de page". On les appelle des **conteneurs sémantiques**, parce qu'ils donnent un vrai sens à ce qu'ils regroupent. Aller on explique mieux à la page qui suit !

Les conteneurs génériques

Un **conteneur générique**, ou **neutre**, comme son nom l'indique, ne dit rien sur ce qu'il contient. Son seul rôle est de créer une boîte pour regrouper des éléments ensemble, sans annoncer leur nature ni leur donner un sens particulier. Il en existe deux : **<div>**, qui regroupe des blocs entiers, et ****, qui cible une petite portion de texte à l'intérieur d'une ligne.

Le conteneur <div>, c'est une balise qui sert simplement à créer une boîte, pour regrouper plusieurs éléments ensemble. Elle n'a pas de sens précis en elle-même, elle ne dit ni "je suis un titre" ni "je suis un texte", son seul travail, c'est de rassembler plusieurs éléments qui vont ensemble, pour pouvoir les traiter comme un seul groupe.

C'est ce qu'on appelle une balise de type **block** : elle prend toute la largeur de l'écran disponible, et se place toujours toute seule sur sa propre ligne,

un peu comme un paragraphe **<p>** ou un titre **<h1>**, qui sont eux aussi des balises **block**. Ça veut dire que si vous mettez plusieurs **<div>** à la suite, elles vont s'afficher dans le navigateur l'une en dessous de l'autre, chacune bien séparée des autres

```
<div>
```

```
  <h2>Qui sommes-nous ?</h2>
```

```
  <p>MIDEESSI est une entreprise technologique.</p>
```

```
</div>
```

**Le conteneur **, c'est une balise qui sert simplement à créer une petite boîte, pour cibler une portion de texte à l'intérieur d'une ligne. Elle n'a pas de sens précis en elle-même, son seul travail, c'est de délimiter un mot ou un groupe de mots précis, pour pouvoir les traiter à part.

C'est ce qu'on appelle une balise de type **inline** : contrairement à **<div>**, elle ne prend pas toute la largeur de l'écran et ne crée aucun saut de ligne elle reste collée au texte autour d'elle, sur la même ligne.

Un exemple d'utilisation de **** :

```
<p>Le riz au poulet coûte <span>1500 FCFA</span>.</p>
```

Ici, **** entoure juste "1500 FCFA", sans casser la phrase. Ça permet ensuite de cibler uniquement ce mot avec du CSS (par exemple, le mettre en rouge ou en gras), sans toucher au reste de la phrase autour.

Les conteneurs sémantiques

Après les conteneurs neutres, nous allons passer aux conteneurs non neutres, ou encore appelés conteneurs sémantiques, c'est-à-dire des conteneurs qui informent le navigateur sur ce qu'ils contiennent. Tous les sites que vous verrez sont segmentés en plusieurs parties : l'en-tête, où se trouvent généralement le menu et le logo ; le bas de page, où l'on retrouve souvent le copyright et les liens secondaires ; et même le contenu principal, qui est lui aussi segmenté en différentes sections.

On pourrait tout regrouper avec **<div>**, puisqu'elle aussi permet de créer ces blocs. Mais le problème, c'est qu'une **<div>** ne dit rien sur ce qu'elle représente : le navigateur voit juste une boîte, sans savoir s'il s'agit de l'en-tête, du menu, ou du pied de page.

C'est là qu'interviennent les conteneurs sémantiques : **<header>**, **<nav>**, **<main>**, **<section>**, **<article>**, **<footer>**. Chacun d'eux joue exactement le même rôle qu'une **<div>**, regrouper du contenu, mais en plus, chacun annonce clairement ce qu'il représente.

Après les conteneurs neutres, nous allons passer aux conteneurs non neutres, ou encore appelés conteneurs sémantiques, c'est-à-dire des conteneurs qui informent le navigateur sur ce qu'ils contiennent. Tous les sites que vous verrez sont segmentés en plusieurs parties : l'en-tête, où se trouvent généralement le menu et le logo ; le bas de page, où l'on retrouve souvent le copyright et les liens secondaires ; et même le contenu principal, qui est lui aussi segmenté en différentes sections.

On pourrait tout regrouper avec **<div>**, puisqu'elle aussi permet de créer ces blocs. Mais le problème, c'est qu'une **<div>** ne dit rien sur ce qu'elle représente : le navigateur voit juste une boîte, sans savoir s'il s'agit de l'en-tête, du menu, ou du pied de page.

C'est là qu'interviennent les conteneurs sémantiques : **<header>**, **<nav>**, **<main>**, **<section>**, **<article>**, **<footer>**. Chacun d'eux joue exactement le même rôle qu'une **<div>**, regrouper du contenu, mais en plus, chacun annonce clairement ce qu'il représente.

Nous avons mentionné plus haut qu'un site est toujours divisé en trois grandes zones : l'en-tête, le pied de page, et le contenu lui-même, ce dernier étant souvent encore divisé en plusieurs parties bien distinctes.

Dans un fichier HTML, chacune de ces zones correspond à un conteneur sémantique précis. Nous allons les voir un par un, en commençant par l'en-tête.

Quand tu ouvres le site de MIDEESSI sur mideessi.com, la toute première bande en haut de l'écran, avec le logo MIDEESSI et le menu de navigation, c'est ce qu'on appelle l'en-tête. C'est la zone que tu vois en premier, avant même de faire défiler la page. En HTML, tu la codes avec **<header>**, et elle indique au navigateur : « voici le début de la page, sa zone d'accueil.»

```
<header>  
  <h1>MIDEESSI</h1>  
</header>
```

Toujours dans cette en-tête, à côté du logo MIDEESSI, tu retrouves les liens "Services", "Apprendre", "Laboratoire", "Articles", "MIDEESSI", c'est ce qu'on appelle le menu de navigation. Il te permet de te déplacer d'une page à une autre sans jamais te perdre sur le site. En HTML, tu regroupes tous ces liens dans une balise sémantique **<nav>**, pour dire clairement au navigateur : « voici l'ensemble des liens qui servent à naviguer. »

```
<nav>  
<ul>  
  <li><a href="#">Services</a></li>  
  <li><a href="#">Apprendre</a></li>  
  <li><a href="#">Laboratoire</a></li>  
  <li><a href="#">Articles</a></li>  
  <li><a href="#">MIDEESSI</a></li>  
</ul>  
</nav>
```

Tu remarqueras deux nouvelles balises utilisées ici, **/** pour la liste et **<a>** pour rendre chaque lien cliquable, on ne les a pas encore vues en détail, elles seront couvertes plus loin dans le cours.

Toujours dans cette en-tête, à côté du logo MIDEESSI, tu retrouves les liens "Services", "Apprendre", "Laboratoire", "Articles", "MIDEESSI", c'est ce qu'on appelle le menu de navigation. Il te permet de te déplacer d'une page à une autre sans jamais te perdre sur le site. En HTML, tu regroupes tous ces liens dans une balise sémantique **<nav>**, pour dire clairement au navigateur : « voici l'ensemble des liens qui servent à naviguer. »

```
<nav>  
  <ul>  
    <li><a href="#">Services</a></li>  
    <li><a href="#">Apprendre</a></li>  
    <li><a href="#">Laboratoire</a></li>  
    <li><a href="#">Articles</a></li>  
    <li><a href="#">MIDEESSI</a></li>  
  </ul>  
</nav>
```

Tu remarqueras deux nouvelles balises utilisées ici, **/** pour la liste et **<a>** pour rendre chaque lien cliquable, on ne les a pas encore vues en détail, elles seront couvertes plus loin dans le cours.

Tu te demandes peut-être pourquoi on ne met pas le `<nav>` dans le **<header>** ? Eh bien, en réalité, c'est exactement ce qui est conseillé : le **<nav>** se place toujours à l'intérieur du **<header>**, parce que le menu fait naturellement partie de l'en-tête, les deux se trouvent toujours ensemble, tout en haut de la page.

Tu te demandes peut-être aussi pourquoi ne pas simplement placer les liens du menu directement dans **<header>**, sans passer par **<nav>** ? Après tout, **<header>** est déjà un conteneur, et rien n'empêche techniquement d'y mettre les liens directement. Le problème, c'est que **<header>** ne dit rien de précis sur ce qu'il contient, il peut regrouper un logo, un slogan, une image, et des liens, tout mélangé. Si tu places tes liens directement dans **<header>** sans **<nav>**, le navigateur ne peut pas identifier clairement lesquels de ces éléments forment un menu de navigation.

C'est pour ça qu'on utilise **<nav>** à l'intérieur de **<header>** : `<nav>` isole précisément la partie qui sert à naviguer, en lui donnant un sens clair, distinct du reste de l'en-tête.

```
<header>
```

```
<h1>MIDEESSI</h1>
```

```
<nav>
```

```
<a href="#">Services</a>
```

```
<a href="#">Apprendre</a>
```

```
<a href="#">Contact</a>
```

```
</nav>
```

```
</header>
```

Tout en bas du site de MIDEESSI, tu retrouves généralement les liens de contact, les icônes des réseaux sociaux, et une mention de copyright, c'est le pied de page, la dernière zone visible du site. En HTML, tu codes cette zone avec `<footer>`, pour indiquer clairement au navigateur qu'on arrive à la fin du document.

```
<footer>
```

```
  <p>© 2026 MIDEESSI — Tous droits réservés</p>  
</footer>
```

On est d'accord qu'un site n'est pas uniquement composé d'un <header> et d'un <footer>, entre les deux se trouve le cœur même du site, là où toutes les informations essentielles sont regroupées. C'est cette zone qu'on appelle le contenu principal, et en HTML, elle se code avec **<main>**.

<main>

<h2>Bienvenue chez MIDEESSI</h2>

<p>Votre partenaire digital à Cotonou.</p>

</main>

Le contenu principal **<main>** d'un site est rarement un seul bloc de texte, il est presque toujours découpé en plusieurs parties, chacune traitant d'un sujet différent. Sur l'accueil du site de MIDEESSI, tu retrouves par exemple une zone "Développer vos activités" et une zone "Formation" : deux sujets distincts, qui méritent d'être séparés. En HTML, chacun de ces blocs thématiques se code avec **<section>**.

```
<main>
```

```
  <section>
```

```
    <h2>Développer vos activités</h2>
```

```
    <p>Sites web & applications, réseaux sociaux</p>
```

```
  </section>
```

```
  <section>
```

```
    <h2>Formation</h2>
```

```
    <p>Formations et livres numériques.</p>  
  </section>  
</main>
```

On peut aussi utiliser `<article>` pour diviser le `<main>`, en plus de `<section>`. La différence, c'est que `<article>` s'utilise quand chaque morceau de contenu a du sens tout seul, même sorti de la page, alors que `<section>` sert juste à regrouper plusieurs éléments qui parlent du même sujet.

```
<main>  
  <section>  
    <h2>Nos services</h2>  
    <article>  
      <h3>Sites web & applications</h3>  
      <p>Création de sites web, applications et  
boutiques en ligne.</p>  
    </article>  
    <article>  
      <h3>Automatisation & IA</h3>  
      <p>Workflows automatisés et outils IA.</p>  
    </article>  
  </section>  
</main>
```

Ici, **`<section>`** regroupe tous les services sous un même thème, et chaque **`<article>`** à l'intérieur représente un service qui pourrait être sorti de cette page affiché seul ailleurs et rester parfaitement compréhensible.

Après avoir vu tout cela, vous savez donc à quoi ressemble la structure totale d'une page web, que ce soit la structure de base ou avec les structures sémantiques.

Les images

Jusqu'ici, on ne manipulait que du texte et des balises. Mais un site n'est pas fait uniquement de texte, il y a aussi des images. La question est donc: comment mettre une image dans un fichier HTML ?

Vous connaissez déjà à connaître la chanson il faut une balise la balise **** en plus exactement . Alors comment on utilise cette balise ? Une idée peut être ? Bon c'est assez simple.

Elle est différente des balises qu'on a vues jusqu'ici parce qu'elle ne s'ouvre pas et ne se ferme pas avec deux balises séparées. Il n'y a pas de **...**. Une seule balise suffit, et elle se termine par elle-même : ****.

La balise **** toute seule ne sert à rien. Il lui faut un attribut obligatoire, **src**, qui veut dire "source". C'est là qu'on indique où se trouve l'image.

Par exemple :

```

```

Mais avant ça, il faut comprendre le chemin, mais concrètement c'est quoi un chemin ?

src ne prend pas n'importe quoi. Il faut lui donner un chemin, c'est-à-dire l'endroit exact où se trouve l'image par rapport à notre fichier HTML. Si le chemin est faux, l'image ne s'affichera jamais, même si le fichier existe bel et bien.

Prenons cette structure de dossier :

```
mon-site/  
├── index.html  
└── logo.png
```

index.html est à la **racine** du dossier mon-site.

logo.png est un niveau plus bas, rangé dans le dossier images.

Pour construire le chemin, on se pose une seule question : où je suis, et où est l'image que je veux atteindre ?

Je suis dans index.html, à la racine. L'image est dans images, qui est juste à côté. Donc je n'ai qu'à descendre dans ce dossier, avec un slash / pour dire "entre dans" :

Les images

Jusqu'ici, on ne manipulait que du texte et des balises. Mais un site n'est pas fait uniquement de texte, il y a aussi des images. La question est donc: comment mettre une image dans un fichier HTML ?

Vous connaissez déjà à connaître la chanson il faut une balise la balise **** en plus exactement . Alors comment on utilise cette balise ? Une idée peut être ? Bon c'est assez simple.

Elle est différente des balises qu'on a vues jusqu'ici parce qu'elle ne s'ouvre pas et ne se ferme pas avec deux balises séparées. Il n'y a pas de **...**. Une seule balise suffit, et elle se termine par elle-même : ****.

La balise **** toute seule ne sert à rien. Il lui faut un attribut obligatoire, **src**, qui veut dire "source". C'est là qu'on indique où se trouve l'image.

Resumé CSS

Le CSS (Cascading Style Sheets, feuilles de style en cascade) est le langage qui permet de définir l'apparence visuelle d'une page HTML : couleurs, tailles, espacements, positionnement, polices. Le HTML décrit la structure du contenu (ce qu'il y a sur la page), le CSS décrit son apparence (comment ça se présente). Sans CSS, une page HTML s'affiche avec un style minimal par défaut du navigateur, sans mise en forme.

Le CSS fonctionne par règles : on cible un ou plusieurs éléments HTML, et on leur applique des styles.

1. Lier le CSS au HTML

Avant qu'une règle CSS puisse s'appliquer, il faut relier le fichier CSS à la page HTML. Il existe 3 méthodes :

- Fichier externe : le CSS est écrit dans un fichier séparé (par exemple style.css), relié au HTML via une balise `<link>` placée dans le `<head>`. C'est la méthode recommandée, car elle sépare clairement structure (HTML) et style (CSS), et permet de réutiliser le même fichier CSS sur plusieurs pages.

```
<head>
```

```
<link rel="stylesheet" href="style.css">
```

```
</head>
```

- Balise `<style>` : le CSS est écrit directement dans le `<head>` du fichier HTML, entre des balises `<style>`. Utile pour un test rapide, mais mélange structure et style dans un seul fichier.

```
<head>
```

```
<style>
```

```
p {
```

```
  color: blue;
```

```
}
```

```
</style>
```

```
</head>
```

- Style inline : le CSS est écrit directement sur un élément HTML, via l'attribut `style`. À éviter en usage général, car ça ne cible qu'un seul élément à la fois et rend le code difficile à maintenir.

```
<p style="color: blue;">Texte</p>
```

2. Le sélecteur

- Un sélecteur est la partie d'une règle CSS qui indique quel(s) élément(s) HTML on veut styliser. C'est la cible de la règle. Il existe plusieurs types de sélecteurs :
- Sélecteur de balise : cible directement un type d'élément HTML, comme **p, h1, div**
- Sélecteur de classe : cible tous les éléments portant une classe précise, s'écrit avec un point devant le nom, comme **.carte**
- Sélecteur d'id : cible un seul élément précis portant cet identifiant, s'écrit avec un dièse devant le nom, comme **#header**

```
.carte {  
  color: red;  
}
```

3. La propriété

Une propriété est la caractéristique visuelle qu'on veut modifier sur l'élément ciblé : la couleur, la taille, l'espacement, la police, etc. Chaque propriété a un nom précis et fixe, défini par le CSS lui-même (on ne peut pas inventer une propriété). Exemples de propriétés : **color**, **background-color**, **font-family**, **padding**, **margin**.

```
p {  
  font-family: Arial;  
}
```

4. La valeur

La valeur est ce qu'on attribue à une propriété : elle précise concrètement le résultat voulu. Chaque propriété accepte certains types de valeurs seulement (une couleur pour **color**, une taille pour **padding**, etc.). La propriété et la valeur sont toujours séparées par deux points, et la règle se termine par un point-virgule.

```
p {  
  font-family: Arial;  
}
```

5. Syntaxe de base

Une règle CSS suit toujours la même structure : un sélecteur, suivi d'accolades contenant une ou plusieurs déclarations. Chaque déclaration associe une propriété et une valeur, séparées par deux points, et se termine par un point-virgule.

```
selecteur {  
  propriete: valeur;  
}
```

Exemple concret, en reprenant les 3 notions vues juste avant :

```
p {  
  color: #1a1a1a;  
  font-family: Arial;  
}
```

Ici, **p** est le sélecteur (on cible tous les paragraphes), **color** et **font-family** sont des propriétés, **#lalala** et **Arial** sont leurs valeurs respectives. On peut mettre plusieurs déclarations dans une même règle, chacune sur sa propre ligne pour la lisibilité.

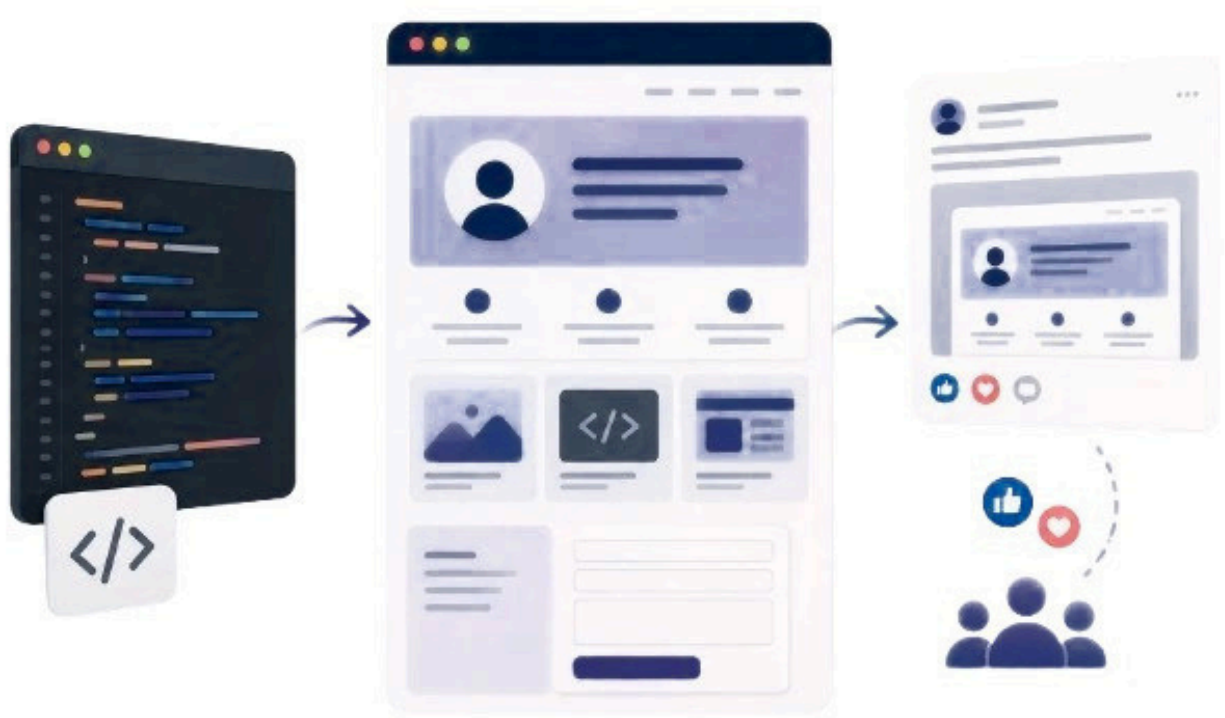
Cette section se place juste avant le modèle de boîte : une fois qu'on sait écrire une règle CSS correctement, on peut passer à ce que ces règles permettent de contrôler concrètement l'espace occupé par chaque élément.

6. Syntaxe de base

Le modèle de boîte décrit comment chaque élément HTML est représenté par le navigateur : sous la forme d'une boîte rectangulaire, composée de plusieurs couches empilées de l'intérieur vers l'extérieur.

- Le contenu (content) : le texte ou l'élément lui-même, avec sa largeur et sa hauteur (width, height)
- Le remplissage (padding) : l'espace intérieur entre le contenu et la bordure de la boîte
- La bordure (border) : le trait qui entoure la boîte
- La marge (margin) : l'espace extérieur entre cette boîte et les éléments voisins

PREMIER PROJET PORTFOLIO



Ceci est votre premier projet, noté sur 100. Veuillez le prendre au sérieux.

Félicitations !

Vous venez de faire un très grand pas dans votre parcours de dev junior.

Vous êtes maintenant capables de créer une page web et de la partager avec vos proches. Ce n'est pas rien et on vous félicite encore une fois pour ça.

Mais maintenant, quand on crée quelque chose, il faut aussi apprendre à le montrer.

Vous ne créez pas pour que votre travail reste uniquement entre vous et nous.

À partir de maintenant, vous allez partager ce que vous construisez.

On commence par partager

Avant de continuer, publiez sur Facebook une courte publication pour annoncer que vous commencez votre parcours en développement Web & Mobile.

Dans votre publication :

- dites ce que vous avez découvert pendant cette première séance ;
- dites ce que vous aimeriez être capable de construire à la fin de la formation ;
- taguez la page MIDEESSI.

Pas besoin d'écrire quelque chose de compliqué.
Écrivez avec vos propres mots.

Et surtout :

Pas d'IA sauf pour corriger peut être des fautes.

Cette publication doit venir de vous.

C'est fait ?

Bon, maintenant, passons dans le vif du sujet.

Construisons votre premier portfolio

Vous allez créer votre premier site web portfolio.

Un portfolio est un site web qui permet à une personne de présenter qui elle est, ce qu'elle apprend, ce qu'elle sait faire et ce qu'elle a déjà réalisé.

Mais cette fois, nous n'allons pas vous donner la structure toute faite.

Vous devez réfléchir à la manière dont votre page doit être organisée.

Votre portfolio doit contenir

Votre identité

Votre nom et votre prénom doivent apparaître clairement sur le site.

Une courte présentation

Présentez-vous en quelques lignes.

Qui êtes-vous ?

Que faites-vous actuellement ?

Pourquoi avez-vous commencé à apprendre le développement Web & Mobile ?

Une photo

Ajoutez une photo simple de vous.

N'oubliez pas de donner à votre image les informations nécessaires pour qu'elle soit correctement comprise par le navigateur.

Votre parcours

Expliquez brièvement votre parcours et ce que vous souhaitez accomplir grâce à cette formation.

Vos compétences

Présentez les notions que vous avez déjà découvertes.

Vous pouvez notamment présenter :

- HTML ;
- les balises ;
- les attributs ;
- les id ;
- les class ;
- les liens ;
- etc.

À vous de choisir comment organiser ces informations.

Votre premier projet

Présentez la première page web que vous avez réalisée.

Expliquez en quelques lignes ce que vous avez construit.

Ajoutez un lien permettant de consulter votre projet.

Vos différents liens

Ajoutez des ancres permettant d'accéder à vos différents espaces.

Par exemple :

- Facebook ;
- GitHub ;
- LinkedIn si vous en avez un ;
- votre adresse email.

Les liens doivent réellement fonctionner.

Votre formulaire de contact

Votre portfolio doit permettre à un visiteur de vous laisser un message.

Le formulaire doit contenir au minimum :

- un champ pour le nom ;
- un champ pour l'adresse email ;
- un champ pour le message ;
- un bouton d'envoi.

Prenez le temps de réfléchir aux éléments HTML nécessaires pour construire ce formulaire.

Organisez votre page

Votre portfolio doit également avoir une navigation. Le visiteur doit pouvoir accéder facilement aux différentes parties de votre page.

Par exemple :

Présentation — Parcours — Compétences — Projets — Contact

Lorsque vous cliquez sur un élément de navigation, vous devez arriver au bon endroit.

C'est ici que vos **id** et vos **ancres** vont réellement servir.

Toutes les notions doivent être utilisées

Ce portfolio est votre exercice de synthèse.

Vous devez donc réutiliser les notions HTML que nous avons vues jusqu'ici :

- la structure html ;
- head ;
- body ;
- title ;
- les titres ;
- les paragraphes ;
- les listes ;
- les images ;

- les ancres ;
- les liens ;
- les attributs ;
- les id ;
- les class ;
- les éléments de formulaire.

Attention

Nous ne cherchons pas à voir combien de balises différentes vous pouvez mettre dans une page.

Nous voulons voir si vous avez compris pourquoi et quand les utiliser.

Avant de coder

Ne commencez pas immédiatement à écrire du HTML.

Prenez quelques minutes pour réfléchir à votre page.

Vous pouvez faire un petit schéma sur une feuille.

Imaginez que quelqu'un découvre votre portfolio pour la première fois.

Qu'est-ce qu'il doit voir en premier ?

Où va-t-il trouver votre présentation ?

Comment va-t-il voir votre premier projet ?

Comment va-t-il vous contacter ?

Comment va-t-il passer d'une partie à une autre ?
Réfléchissez d'abord. Construisez ensuite.

Maintenant, mettez-le en ligne

Votre portfolio ne doit pas rester uniquement sur votre ordinateur.

Vous allez maintenant déployer votre site avec Render.

À la fin de l'exercice, votre portfolio doit être accessible publiquement avec une adresse de la forme :
prenom-nom.onrender.com

Votre prénom et votre nom doivent donc être utilisés dans le nom de votre service Render.

Le lien doit être accessible depuis un navigateur.

Vérification finale

Avant de considérer l'exercice comme terminé, vérifiez votre site.

Votre portfolio contient-il :

- votre nom et votre prénom ;
- votre présentation ;
- votre parcours ;
- votre photo ;
- vos compétences ;
- votre premier projet ;
- des ancres ;
- des liens externes ;
- un formulaire de contact ;
- des id pertinents ;
- des class pertinentes ;
- une structure HTML complète ?

Et surtout :

- les liens fonctionnent ;
- les ancres fonctionnent ;
- la photo s'affiche ;
- le formulaire est correctement affiché ;
- le site est accessible sur Render ;
- l'adresse respecte le format demandé.

Dernière étape : partager

Vous avez maintenant quelque chose que vous avez réellement construit.

Alors ne le gardez pas pour vous.

Publiez votre portfolio sur Facebook.

Votre publication doit :

1. présenter brièvement ce que vous venez de construire ;
2. contenir le lien vers votre portfolio ;
3. taguer MIDEESSI.

Et voilà.

Vous êtes partis d'un fichier HTML.

Vous avez construit votre première page personnelle.

Vous avez appris à structurer des informations, ajouter des images, créer des liens, naviguer dans une page et construire un formulaire.

Et maintenant...

quelqu'un d'autre peut ouvrir votre adresse et voir ce que vous avez construit.

C'est votre premier portfolio.

Gardez-le.

Vous allez le revoir évoluer tout au long de votre parcours.

